# While loops

The four parts of a loop, trace tables pass by pass, counters and accumulators, and digit loops with // and %.

**THE QUESTION**

**How can a program repeat the same steps many times without writing them out again?**

**THE INTUITION**

**A `while` loop is an `if` that goes back: it tests, runs its indented block, then returns to test again. It stops the first time the test is False, so the block must change something the test looks at.**

**KEY TAKEAWAY**

**Every loop needs a start value, a test, a body and an update that moves the test toward False. Trace the first and the last pass by hand.**

num = 1 → test num <= n → True: run the body → num = num + 1 → back to the test → False: carry on after the loop

## 1 `while` repeats its block while the test is True

loop1.py

```
1 n = 5
2
3 num = 1
4 while num <= n:
5     print(num)
6     num = num + 1
7
8 print("Done")
```

output

```
1
2
3
4
5
Done
```

test ran 6 times, body 5

loop-if.py

```
1 n = 5
2
3 num = 1
4 if num <= n:
5     print(num)
6     num = num + 1
7 print("Done with if")
```

output

```
1
Done with if
```

test ran once, body once

A **loop** runs a block of code again and again. The header is `while`, a condition and a colon, as with `if` (see the conditionals sheet); the indented lines are the **body**. After the body Python goes back and tests again. One run of the body is a **pass** (an **iteration**). These two course files differ in one word.

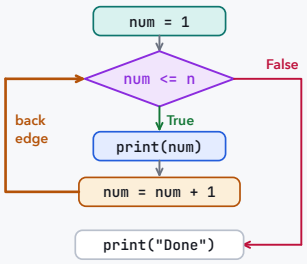
## 2 Every loop has four parts

the notebook's loop

```
n = 5
num = 1                    initialise
while num <= n:            test
    print(num)             body
    num = num + 1          update
print("Done")
```

the body can be many indented lines

prints 1 2 3 4 5 Done




after the loop: runs once

**Initialise** the **loop variable** before the loop. The **test** runs before every pass. The **body** does the work. The **update** changes the loop variable so the test can turn False. The **back edge** from the update to the test makes it a loop. Any step works: `num = num + 2` prints 1 3 5 7 9 11, and `2 ** num` for `num` from 0 to 5 gives 1 2 4 8 16 32.

## 3 A trace table follows the loop pass by pass

countdown.py

```
1 n = 3
2 while n > 0:
3     print(n)
4     n = n - 1
5 print("Liftoff!")
```

| test | n | n > 0 | prints | n after |
|---|---|---|---|---|
| 1 | 3 | True | 3 | 2 |
| 2 | 2 | True | 2 | 1 |
| 3 | 1 | True | 1 | 0 |
| 4 | 0 | False | | exit |

output 3 2 1 Liftoff!

4 tests, 3 passes: the last test is False and the loop ends

A **trace table** has one row per test: the values then, the test's result, what the pass printed and the value after the update. The body runs 3 times but the test runs 4 times; the last test is the False one. After the loop n is 0 and Python goes on to line 5.

## 4 A counter counts; an accumulator collects

the notebook's sum

```
1 s = 0
2 nat = 1
3 n = 4
4 while nat <= n:
5     s = s + nat
6     nat = nat + 1
7 print(s)
```

| test | nat | nat <= n | s after | nat after |
|---|---|---|---|---|
| 1 | 1 | True | 0+1=1 | 2 |
| 2 | 2 | True | 1+2=3 | 3 |
| 3 | 3 | True | 3+3=6 | 4 |
| 4 | 4 | True | 6+4=10 | 5 |
| 5 | 5 | False | 10 | exit |

print(s) shows 10; n * (n + 1) // 2 = 10 agrees

factorial, n = 5: p = p * nat, p after each pass

| | | | | | | |
|---|---|---|---|---|---|---|
| p = 1 at the start | 1 | 2 | 6 | 24 | 120 | 5 × 4 × 3 × 2 × 1 = 120 |
| p = 0 at the start | 0 | 0 | 0 | 0 | 0 | 0 times anything is 0 |

nat is a **counter**: it goes up by 1 each pass. s is an **accumulator**, a running total that each pass adds to. Set it before the loop: a sum starts at 0, a product at 1. The notebook's factorial of 10 gives 3628800. Start p at 0 and every product stays 0.

## 5 `% 10` reads the last digit; `// 10` drops it

digits.py

```
1 n = 4072
2 count = 0
3 total = 0
4 while n > 0:
5     digit = n % 10
6     total = total + digit
7     count = count + 1
8     n = n // 10
9 print(count, total)
```

| test | n | n % 10 | total | count | n // 10 |
|---|---|---|---|---|---|
| 1 | 4072 | 2 | 2 | 1 | 407 |
| 2 | 407 | 7 | 9 | 2 | 40 |
| 3 | 40 | 0 | 9 | 3 | 4 |
| 4 | 4 | 4 | 13 | 4 | 0 |
| 5 | 0 | | 13 | 4 | exit |

prints 4 13: 4 digits, 4 + 0 + 7 + 2 = 13

% 10 reads the last digit; // 10 drops it

For a whole number n, `n % 10` is its last digit and `n // 10` is n without it (see the variables sheet). Repeat until n reaches 0: `count` counts the passes, `total` adds the digits. One pass per digit, so the count agrees with `len(str(4072))`, 4. The loop uses up n: copy it first if you need it later.

## 6 Reverse a number: shift left, then add a digit

reverse.py

```
1 n = 1234
2 rev = 0
3 while n > 0:
4     rev = rev * 10 + n % 10
5     n = n // 10
6 print(rev)
```

| test | n | n % 10 | rev after | n after |
|---|---|---|---|---|
| 1 | 1234 | 4 | 0*10+4=4 | 123 |
| 2 | 123 | 3 | 4*10+3=43 | 12 |
| 3 | 12 | 2 | 43*10+2=432 | 1 |
| 4 | 1 | 1 | 432*10+1=4321 | 0 |
| 5 | 0 | | 4321 | exit |

prints 4321

1200 gives 21; 121 gives 121

`rev * 10` moves the digits built so far one place left; `+ n % 10` adds the next digit on the end. 1200 gives 21: an int has no leading zeros. A number equal to its reverse, such as 121, is a **palindrome**; to test for one, save n first, as the loop ends with n at 0.

# Zero passes, one pass too many, no end

Loops that never start, the < versus <= slip, infinite loops and how to stop them, and reading input until a stop value.

## 7 A loop can run zero times

| program | first test | result | prints | verdict |
|---|---|---|---|---|
| **sum**<br>`nat = 1, n = 0` | `nat <= n` | `False` → skip | `0` | **right**<br>sum of nothing |
| **countdown**<br>`n = 0` | `n > 0` | `False` → skip | `Liftoff!` | **right**<br>nothing to count |
| **digit count**<br>`n = 0` | `n > 0` | `False` → skip | `0 0` | **wrong**<br>0 has one digit |

fix for the digit count: before the loop, set count = 1 if n == 0 (then 0 gives 1)

`while` tests before the first pass. If that test is False, the body never runs and Python goes straight to the line after the loop. Right for a sum or a countdown, wrong for the digit count. Test the smallest input.

## 8 < or <=: one pass apart

`while num <= n:` — n = 5; tests: 6; passes: 5

| num=1 | num=2 | num=3 | num=4 | num=5 | num=6 |
|---|---|---|---|---|---|
| True | True | True | True | True | False |
| 1 | 2 | 3 | 4 | 5 | |

prints 1 2 3 4 5

`while num < n:` — n = 5; tests: 5; passes: 4

| num=1 | num=2 | num=3 | num=4 | num=5 | num=6 |
|---|---|---|---|---|---|
| True | True | True | True | False | not tested |
| 1 | 2 | 3 | 4 | | |

prints 1 2 3 4

num = 5: the edge case

An **off-by-one error** is one pass too many or too few. With `n = 5`, `num <= n` lets 5 in; `num < n` stops there. In the sum of 1 to 4, < gives 6 instead of 10. Check two passes by hand: the first (is the start value right?) and the last (does the final value pass?).

## 9 An infinite loop never finds its test False

bug 1: no update

```
1 num = 1
2 while num <= 5:
3     print(num)
```

prints 1, then 1, then 1 ...
more than 1,000,000 lines of 1 in one second (sent to a file), until Ctrl+C stopped it

bug 2: the test never becomes False

```
1 num = 1
2 while num != 10:
3     num = num + 2
```

num at each test: 1 3 5 7 9 11 13 ...

9 jumps to 11: num is never exactly 10

python3 loop.py, Ctrl+C after 1 second

```
^CTraceback (most recent call last):
  File ".../loop.py", line 2, in <module>
    while num != 10:
          ^^^^^^^^
KeyboardInterrupt
```

guard: `while num != 10 and steps < 100:` stops, prints 201 100

An **infinite loop** runs until you stop it. Press **Ctrl+C** in the terminal (in Jupyter, the stop button): Python raises **KeyboardInterrupt**, shows the line it was on and ends. Fix the test (`num < 10` ends at 11), or add a **guard**, a step count that caps the passes.

## 10 Read input until a sentinel

```
1 total = 0
2 count = 0
3 x = int(input("Next: "))
4 while x != 0:
5     total = total + x
6     count = count + 1
7     x = int(input("Next: "))
8 print(count, "numbers, total", total)
```

terminal: typed 5, 8, 2, 0

```
Next: 5
Next: 8
Next: 2
Next: 0
3 numbers, total 15
```

teal: first read, before the loop
amber: next read, end of the body

x at each test: 5 8 2 0

type 0 first: no passes,

```
0 numbers, total 0
```

A **sentinel** is a value that means "stop", here 0. Read the first value before the loop and the next one as the last line of the body, so the 0 is tested but never added. Leave out the second read and x never changes: an infinite loop. `int()` converts the typed text (see the input and output sheet).

## 11 Common mistakes: what you wrote, what Python says, the fix

| you wrote | the last line Python prints, or what you see; why, and the fix |
|---|---|
| `while count < 3:`<br>`    count = count + 1` | `NameError: name 'count' is not defined. Did you mean: 'round'?`<br>create the name before the loop: `count = 0`, then it prints 3 |
| `n = input("n: ")`<br>`while n > 0:` | `TypeError: '>' not supported between instances of 'str' and 'int'`<br>`input()` gives a str; use `n = int(input("n: "))` |
| `while i <= 3:`<br>`    total = 0` | `prints: 3`<br>the reset runs on every pass; put `total = 0` before the loop and it prints 6 |
| `    n = n / 10` | `prints: 328 14.444444444444427`<br>/ makes a float that shrinks for hundreds of passes before it reaches 0; // prints 4 13 |
| `    print(i)`<br>`i = i + 1` | `prints 1 forever; after Ctrl+C: KeyboardInterrupt`<br>the update is outside the body; indent it and it prints 1 2 3 |

## 12 Try it (answers in green)

**1.** `k = 20; while k > 0: k = k - 6.` Final k? Passes?
-4, after 4 passes.

**2.** `x = 1; while x < 100: x = x * 3.` Final x?
243: 81 still passed the test.

**3.** Product of the digits of 234 with a loop?
24: `p = 1, p = p * (n % 10)`.

**4.** Sum `i` from 1 `while i < 10:`. The total?
45: 10 is left out; <= gives 55.

**5.** `n = 7; while n != 0: n = n - 2.` Does it stop?
No: 5, 3, 1, -1, -3, -5 ... skips 0.

**6.** How many `n = n // 2` steps take 1000 to 0?
10 (the last one takes 1 to 0).

**start value · test · body · update toward False · trace the first and last pass · Ctrl+C stops a runaway loop**

Source: the while loop notebook and the scripts loop1.py and loop-if.py, nipunbatra.github.io/comp22. Every output, trace value and error message comes from running the code with Python 3.11.11, with typed input fed in and infinite loops stopped by Ctrl+C (SIGINT) after one second. Next: the break, continue and nested loops sheet.